

복약기 — 화면별 API 명세서

노인 복약 관리 시스템 · MVP

작성	2026-08-24
기준 문서	spec.md §8 (최종 계약서) · api-decisions.md
결정 이유	api-worksheet.md
구성	보호자 앱용 8개 + 디바이스용 2개 = 엔드포인트 10개
공통 객체	Device · Dose · Medication · Notification

▶ 목차

- 01 공통 규칙

02 공통 객체 4개

03 화면 1 — 로그인 / 회원가입

04 화면 2 — 기기 목록

05 화면 3 — 기기 등록

06 화면 4 — 기기 상세 (이번 주)
- 07 화면 5 — 약 설정

08 화면 6 — 복약 기록

09 화면 7 — 알림 목록

10 공용 팝업 — 복약 상세

11 디바이스용 API

12 접근 제어

읽는 순서 — 각 페이지 왼쪽은 구현된 화면, 오른쪽은 그 화면이 호출하는 API다. 응답 JSON 은 실제 값이 든 예시이므로 그대로 목업으로 써도 된다.

▶ 기본

Base URL	/api (개발 http://localhost:3000/api)
앱 인증	Authorization: Bearer <Supabase access token>
디바이스 인증	X-Device-Key: <기기 API 키>
형식	application/json
모든 시각	UTC ISO 8601 "2026-08-23T23:00:00Z" KST 변환은 화면이 한다
예외	Medication.time 만 백시계 값 "08:00"
모든 id	문자열 (JS 숫자 정밀도 문제 회피)
전화번호	숫자만 "01012345678" · 하이픈은 화면이 붙인다

▶ 응답 모양 규칙

- 응답 최상위는 항상 객체로 감싼다
- 하나만 돌려줄 때도 배열에 담는다 — { "devices": [{ ... }] }
→ 클라이언트 코드가 항상 같다. map() 이 어디서나 안전
- "없음" 은 null 이 아니라 빈 배열 — { "medications": [] }
→ null 체크가 사라진다
- 경로가 가리키는 자원 자체가 없으면 404 / 그 자원의 부속물이 없으면 200 + []
- 자원은 경로, 거르는 조건은 쿼리스트링

▶ 오류 응답 — 표준이 없으므로 프로젝트에서 정한다

```
{
  "error": {
    "type": "validation_error",
    "code": "INVALID_CODE",
    "message": "등록코드가 일치하지 않습니다"
  }
}
```

message	사용자가 읽는다. 화면 오류 때에 그대로 출력
code	프로그램 — 정확히 이 오류일 때만 분기
type	프로그램 — 비슷한 오류를 묶어 같은 대응

▶ type — 프론트가 어떻게 대응해야 하는가

auth_error	로그인 화면으로 보낸다
permission_error	목록으로 되돌리고 안내
validation_error	그 화면에 머물며 입력을 고치게
state_error	이미 처리됐거나 사라짐 → 화면을 새로 불러온다

▶ 상태 코드 (422 는 쓰지 않는다)

HTTP	code	언제
400	VALIDATION_ERROR	필수값 누락, 형식 오류
400	SERIAL_NOT_FOUND	그런 일련번호 없음
400	INVALID_CODE	등록코드 불일치
400	DOSE_NOT_DUE	아직 오지 않은 복용 건
401	UNAUTHORIZED	토큰 없음·만료
401	INVALID_DEVICE_KEY	기기 API 키 틀림
403	FORBIDDEN	내 기기/내 알림이 아님
404	DEVICE_NOT_FOUND	경로의 기기 id 없음
404	NOT_FOUND	경로의 복용 건·알림 id 없음
409	ALREADY_REGISTERED	이미 등록된 기기
409	ALREADY_TAKEN	이미 복용 처리됨

401 과 403 — 401 은 "출입증을 안 보여줬다"(인증), 403 은 "출입증은 봤는데 이 층은 못 들어간다"(인가)

▶ Device

```
{
  "id": "d3f1a2b4-9c8e-4f21-b7a3-0e5d6c1f8a92",
  "serial": "PB-2026-0001",
  "nickname": "할아버지 기기",
  "patient_phone": "01012345678",
  "timezone": "Asia/Seoul"
}
```

registration_code · device_api_key 는 절대 내려주지 않는다.

등록코드가 새면 "기기를 만져본 사람만 안다" 는 전제가 깨지고, API 키가 새면 curl 로 taken 거짓 보고를 보낼 수 있다.

DB 컬럼 목록 ≠ API 응답 필드 목록.

- patient_phone — 없으면 null. 있을 때만 화면 4 에 [전화 걸기]
- timezone — 어느 화면도 안 쓴다. 서버가 "매일 08:00" 을 UTC 로 바꿀 때 쓰는 내부 계산용

▶ Medication — 약 설정 = 시간표

```
{
  "id": "31",
  "device_id": "d3f1a2b4-...",
  "name": "혈압약",
  "dosage": "1정",
  "time": "08:00",
  "days": [0, 1, 2, 3, 4, 5, 6]
}
```

★ time 은 UTC 가 아니다 — 시각 규칙의 유일한 예외

"매일 08:00" 은 순간이 아니라 규칙이라 UTC 로 바꿀 수 없다. UTC 로 바꾸려면 "어느 날의 08시" 인지가 있어야 하는데 매일 반복되는 값에는 날짜가 없다.

```
medications.time "08:00" + devices.timezone
↓ 서버가 그날의 복약 건을 만들 때 계산
doses.scheduled_at "2026-08-23T23:00:00Z"
```

시간표는 벽시계, 출석부는 UTC.

- days — 0=일 ~ 6=토 (JS Date.getDay() 와 같은 번호)
- 기기당 1줄이므로 배열에 0개 또는 1개뿐

▶ Dose — 복약 건 = 출석부 한 줄

```
{
  "id": "8821",
  "device_id": "d3f1a2b4-...",
  "scheduled_at": "2026-08-23T23:00:00Z",
  "status": "missed",
  "notified_at": "2026-08-23T23:00:00Z",
  "dispensed_at": null,
  "taken_at": null,
  "taken_source": null
}
```

status	scheduled · notified · dispensed · taken · missed
taken_source	ir · camera · manual · null

시각 4개를 항상 다 내려준다. missed 가 두 종류인 것을 화면이 구분해야 한다.

dispensed_at 이 **비었는데** missed → × 약을 안 꺼내셨어요

dispensed_at 이 **있는데** missed → △ 꺼내셨는데 아직 안 드셨어요

▶ Notification

```
{
  "id": "412",
  "device_id": "d3f1a2b4-...",
  "dose_id": "8821",
  "type": "missed_not_dispensed",
  "message": "약을 안 꺼내셨어요",
  "created_at": "2026-08-23T23:05:00Z",
  "read_at": null
}
```

type	missed_not_dispensed (안 꺼냄) missed_not_taken (꺼냈으나 미복용)
read_at	안 읽었으면 null

message 는 서버가 type 으로부터 만든다. DB 에 저장하지 않는다.

· 저장 안 하는 이유 — 문구를 고치면 과거 알림까지 같이 바뀔다

· 서버가 만드는 이유 — 나중에 FCM 푸시도 같은 함수를 써야 **목록과 푸시 문구가 어긋나지 않는다**

기기 별명과 복약 건은 **여기 안 담는다**. devices · doses 배열을 따로 주고 화면이 id 로 짝을 맞춘다. 같은 기기 알림이 여러 개일 때 별명이 중복해서 실리는 것을 막고, 나중에 알림 목록에 기기 아이콘·전화 버튼을 넣어도 서버를 안 고친다.



복약기

부모님의 복약을 함께 챙깁니다

이메일
example@email.com

비밀번호
6자 이상

로그인

회원가입

— 또는 —
소셜 로그인은 추후 제공됩니다

서버 API 없음

Supabase Auth 를 프론트에서 직접 호출한다

▶ 왜 우리 서버를 거치지 않나

- 회원가입·로그인·토큰 발급·만료 관리를 **Supabase Auth** 가 통째로 담당한다
- 우리 서버(Express)는 **발급된 토큰을 검증**하기만 한다

```
// 로그인 (프론트에서 직접)
const { data, error } = await supabase.auth
  .signInWithPassword({ email, password })

// 이후 모든 API 호출에 이 토큰을 헤더로 실어 보낸다
fetch('/api/devices', {
  headers: { Authorization: `Bearer ${data.session.access_token}` }
})
```

▶ 서버가 토큰으로 하는 일

- ① Authorization 헤더에서 토큰을 꺼낸다
- ② 서명을 검증한다 (Supabase 가 발급한 것인지)
- ③ 토큰 안의 user_id 를 얻는다
- ④ 그 user_id 로 DB 를 조회한다

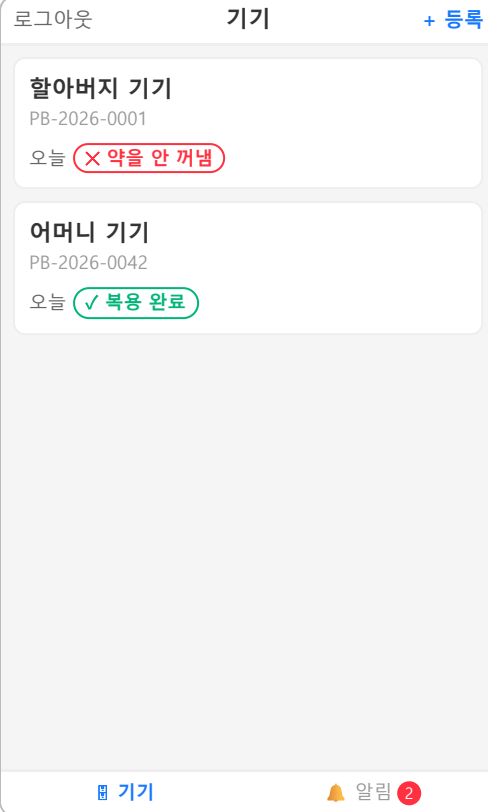
```
SELECT d.* FROM devices d
JOIN user_devices ud ON ud.device_id = d.id
WHERE ud.user_id = '<토큰에서 얻은 값>' ← "내 것" 이 걸려진다
```

클라이언트는 **"누구의 데이터를 달라" 고 말하지 않는다**. 말할 수 있게 두면 남의 user_id 를 적어 보내는 순간 뚫린다. 누구인지는 오직 토큰만이 말한다. 호텔에 비유하면 — 프론트에 "302호 열쇠 주세요" 라고 말하는 게 아니라, 카드키를 대면 기계가 알아서 그 사람 방만 열어주는 것.

▶ 화면이 받는 props (퍼블리싱 계약)

loading	요청 중 (버튼 비활성 + 스피너)
error	오류 문구 (없으면 null)
onLogin(values)	{ email, password }
onSignup(values)	{ email, password }

화면 검증은 **형식까지만** — 빈칸, 이메일 모양, 6자 이상. "이미 가입된 이메일" 은 서버만 알 수 있으므로 error 로 받아 표시한다.



4가지 상태 모두 구현 — 정상 / 비어있음(등록 버튼) / 로딩(스켈레톤) / 오류(다시 시도)

GET /api/devices

화면 열릴 때 · 당겨서 새로고침 · 30초 폴링 — 모두 같은 호출

요청

GET /api/devices?limit=20&offset=0
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...

- limit / offset 둘 다 선택 (기본 20 / 0)
- 정렬은 별명 가나다순 고정 — 쿼리로 못 바꾼다

응답 200

```
{
  "devices": [
    { "id": "d3f1a2b4-...", "serial": "PB-2026-0001",
      "nickname": "할아버지 기기",
      "patient_phone": "01012345678", "timezone": "Asia/Seoul" },
    { "id": "a7c2e5f8-...", "serial": "PB-2026-0042",
      "nickname": "어머니 기기",
      "patient_phone": null, "timezone": "Asia/Seoul" }
  ],
  "today_doses": [
    { "id": "8821", "device_id": "d3f1a2b4-...",
      "scheduled_at": "2026-08-23T23:00:00Z", "status": "missed",
      "notified_at": "2026-08-23T23:00:00Z", "dispensed_at": null,
      "taken_at": null, "taken_source": null },
    { "id": "9104", "device_id": "a7c2e5f8-...",
      "scheduled_at": "2026-08-24T00:30:00Z", "status": "taken",
      "notified_at": "2026-08-24T00:30:00Z",
      "dispensed_at": "2026-08-24T00:31:00Z",
      "taken_at": "2026-08-24T00:33:00Z", "taken_source": "camera" }
  ],
  "unread_count": 2,
  "has_more": false
}
```

설계 요점

- 배열을 나눠 주고 화면이 device_id 로 짝을 맞춘다. 카드 한 장에 devices + doses 두 테이블이 섞여 있다
- today_doses — 기기 타임존 기준 오늘의 복약 건. 오늘이 복용 요일이 아니거나 약 미설정이면 그 기기의 줄이 없다
- unread_count — 하단 탭 배지. 별도 API 없이 여기에 넣는다. 배지가 보이는 화면은 2·7 둘뿐이고, 폴링할 때 카드도 같이 갱신되는 이득이 있다
- offset 페이지네이션인 이유 — 기기 목록은 보호자가 직접 등록할 때만 늘어나 순번이 밀릴 일이 없다. 정렬이 별명 가나다순이라 커서로 쓸 값(별명)이 중복 가능하고 한글 정렬 순서가 DB 설정에 따라 달라진다
- 빈 상태 — 200 + 두 배열 모두 [], unread_count: 0 (404 아님)

←

기기 등록

등록 정보는 어디에 있나요?

일련번호와 등록코드는 기기 뒷면 스티커에 적혀 있습니다.

일련번호

PB-2026-0001

등록코드

A1B2C3

기기 별명 선택

할아버지 기기

등록하기

취소

열릴 때 서버를 부르지 않는다 — 빈 폼이다.

POST /api/user-devices

[등록하기] 를 눌렀을 때

경로가 /api/devices 가 아닌 이유 — 이 API 가 만드는 것은 **기기가 아니라 연결**이다. 기기(PB-2026-0001)는 공장에서 이미 만들어져 DB 에 있고, 새로 생기는 것은 **user_devices** 의 한 줄이다. devices 테이블은 손대지 않는다.
나중에 "등록 해제" 를 만들 때도 DELETE /api/user-devices/:id 가 **"연결을 끊는다"** 로 정확히 읽힌다.

요청

```
POST /api/user-devices
Authorization: Bearer eyJhbGciOi...
```

```
{
  "serial": "PB-2026-0001",
  "code": "A1B2C3",
  "nickname": "할아버지 기기"
}
```

응답 201 Created

```
{
  "devices": [
    { "id": "d3f1a2b4-...", "serial": "PB-2026-0001",
      "nickname": "할아버지 기기",
      "patient_phone": null, "timezone": "Asia/Seoul" }
  ]
}
```

실패 응답

HTTP	code	message (화면에 그대로 출력)
400	VALIDATION_ERROR	일련번호를 입력해 주세요
400	VALIDATION_ERROR	등록코드는 영문·숫자 6자리입니다
400	SERIAL_NOT_FOUND	해당 일련번호의 기기를 찾을 수 없습니다
400	INVALID_CODE	등록코드가 일치하지 않습니다
409	ALREADY_REGISTERED	이미 등록된 기기입니다

서버 처리

- ① devices 에서 serial 로 기기를 찾는다
(대소문자·공백·하이픈을 무시하고 비교)
- ② code 가 맞는지 확인한다
- ③ user_devices 에 (user_id, device_id) 한 줄 추가

왜 등록코드가 필요한가 — 일련번호는 PB-2026-0001, 0002 ... **순차적**이다. 코드가 없다면 옆 번호를 찍어 남의 기기를 등록하고, 복약 기록을 보거나 **복용 시간을 바꿔버릴** 수 있다.
일련번호는 **공개돼야 쓸모 있는 값**(AS 문의 시 불러줌)이고, 등록코드는 **기기를 실제로 가진 사람만 아는 값**이다.

서버는 클라이언트를 믿지 않는다. 화면이 trim().toUpperCase() 를 하지만 서버도 반드시 해야 한다 — cur1 로 직접 보낼 수 있다.
화면의 정리는 사용자 편의, 서버의 정리는 필수.

←

할아버지 기기 · 이번주

주간 현황

17일 ~ 23일

월	화	수	목	금	토	일
17	18	19	20	21	22	23
✓	✓	⚠	✗	✓	✓	✗

✓ 복용 ✕ 안 꺼냄 ⚠ 미복용 ⌚ 진행중 · 예정

약 설정

복약 기록

전화 걸기 (010-1234-5678)

칸을 탭하면 **공용 팝업**(10페이지)이 열린다. 오늘 칸은 열은 파란 배경.

GET /api/devices/:id/doses

★ 화면 6(복약 기록)과 같은 API — 기간만 다르게 부른다

요청

```
GET /api/devices/d3f1a2b4-9c8e-4f21-b7a3-0e5d6c1f8a92/doses
?from=2026-08-17&to=2026-08-23
Authorization: Bearer eyJhbGci...
```

경로의 `:id` 는 UUID (Device.id). `serial` 이 아니다.
`serial` 은 순차적이라 URL 에 쓰면 옆 번호를 찍어볼 수 있다.

응답 200

```
{
  "devices": [
    { "id": "d3f1a2b4-...", "serial": "PB-2026-0001",
      "nickname": "할아버지 기기",
      "patient_phone": "01012345678", "timezone": "Asia/Seoul" }
  ],
  "doses": [
    { "id": "8821", "device_id": "d3f1a2b4-...",
      "scheduled_at": "2026-08-23T23:00:00Z", "status": "missed",
      "notified_at": "2026-08-23T23:00:00Z", "dispensed_at": null,
      "taken_at": null, "taken_source": null },
    { "id": "8820", "device_id": "d3f1a2b4-...",
      "scheduled_at": "2026-08-22T23:00:00Z", "status": "taken",
      "notified_at": "2026-08-22T23:00:00Z",
      "dispensed_at": "2026-08-22T23:02:00Z",
      "taken_at": "2026-08-22T23:03:00Z", "taken_source": "ir" }
    // ... 이 기간에 존재하는 복약 건만
  ],
  "medications": [
    { "id": "31", "device_id": "d3f1a2b4-...", "name": "혈압약",
      "dosage": "1정", "time": "08:00", "days": [0,1,2,3,4,5,6] }
  ],
  "has_more": true
}
```

설계 요점

- `doses` 는 7개가 아닐 수 있다. 복용 요일이 월·수·금이면 3개다. 화면이 자기가 보낸 `from~to` 로 날짜 7칸을 먼저 그리고 각 날짜의 복약 건을 찾아 끼운다. 없는 칸은 점선
- `devices` 를 같이 담는 이유 — 화면이 별명·전화번호를 쓴다. 라우터를 붙이면 이 화면으로 바로 들어오거나 새로고침할 수 있으므로 "화면 2 를 거쳐 왔다" 고 가정하면 안 된다
- `medications` — 약 미설정 구분용
 [] → 약을 아직 설정 안 함 → "약 설정하기" 버튼
 [{...}] → 설정은 있는데 이번 주에 복용 요일이 없음 → 그냥 빈 주
- 날짜 범위로 자르므로 `limit-before-offset` 이 필요 없다. 8/17~8/23 은 새 기록이 쌓여도 같은 구간이라 순번이 밀리는 문제가 없다 — 커서 방식의 장점을 그냥 얻는다
- 빈 상태 — 기기가 없으면 404 `DEVICE_NOT_FOUND` / 복약 건이 없으면 200 + `"doses": []`

날짜를 프론트가 계산하는 것은 MVP 한정. spec.md §3.4 가 "화면에는 KST 로 표시" 라고 못박아 이 앱은 한국 전용이므로 브라우저 시계와 서버 시계가 일치한다. 해외 거주 보호자를 지원하게 되면 — API 모양은 그대로 두고 `from/to` 가 없을 때 서버가 기기 타임존 기준 기본값을 채우면 된다.

←

약 설정

할아버지 기기

약 이름

혈압약

용량

1정

복용 시간

08:00

복용 요일

월

화

수

목

금

토

일

저장하면 다음 복용분부터 적용됩니다. 이미 지난 기록은 바뀌지 않습니다.

저장

GET /api/devices/:id/medications

화면 열릴 때

```
{
  "devices": [ { "...Device..." } ],
  "medications": [
    { "id": "31", "device_id": "d3f1a2b4-...", "name": "혈압약",
      "dosage": "1정", "time": "08:00", "days": [1,2,3,4,5] }
  ]
}
```

약을 아직 설정 안 했으면 200 + "medications": [] — 404 가 아니다. 새 기기는 "없는 게 정상" 이지 오류가 아니다. 프론트가 오류 분기를 안 만들어도 된다.

PUT /api/devices/:id/medications

[저장] 을 눌렀을 때

```
// 요청
{
  "name": "혈압약",
  "dosage": "1정",
  "time": "08:00",
  "days": [1, 2, 3, 4, 5]
}

// 응답 200
{ "medications": [ { "...Medication..." } ] }
```

★ POST 가 아니라 PUT 인 이유 — 명등

POST	"새로 만들어라" → 두 번 보내면 두 개가 생길 수 있다
PUT	"이 주소를 이 내용으로 만들어라" → 몇 번 보내도 결과가 같다

약 설정은 기기당 1줄이고 저장은 항상 덮어쓰기다. 저장 버튼을 두 번 눌러도, 네트워크가 느려 요청이 두 번 나가도 결과가 같아야 한다. PostgreSQL 의 ON CONFLICT (device_id) DO UPDATE 로 구현한다.

▶ 실패 응답

HTTP	code	언제
400	VALIDATION_ERROR	약 이름·용량 빈칸, 시각 형식 오류, 요일 0개
403	FORBIDDEN	내 기기가 아님
404	DEVICE_NOT_FOUND	그런 기기 없음

저장 이후의 복용 건부터 적용된다. 이미 만들어진 과거 건은 바뀌지 않는다. 시간표(medication)를 고치는 것이지 출석부(dose)를 고치는 것이 아니다.

화면 구현 참고 — 복용 시간은 antd-mobile 의 Picker(시/분 2칸), 요일은 Selector(multiple) 로 만들었다. DatePicker 는 연도부터 칸이 누적돼 "시각만" 고를 수 없다.

←복약 기록

할아버지 기기 · 최근 30일

8월 23일 (일)

예정 08:00

✕ 약을 안 꺼냄 08:00

8월 22일 (토)

예정 08:00

✓ 복용 완료 08:03

8월 21일 (금)

예정 08:00

✓ 복용 완료 08:05

8월 20일 (목)

예정 08:00

⚠ 꺼냈으나 미복용

8월 19일 (수)

예정 08:00

✓ 복용 완료 08:12

더 보기

항목을 탭하면 공용 팝업이 열린다 — 지난 날짜의 수동 체크도 여기서.

GET /api/devices/:id/doses

★ 화면 4 와 같은 API. 응답 모양도 같다 (6페이지 참고)

▶ 같은 API 를 기간만 다르게 부른다

화면 4

?from=2026-08-17&to=2026-08-23

이번 주 월~일

화면 6

?from=2026-07-25&to=2026-08-23

최근 30일

[더 보기]

?from=2026-06-25&to=2026-07-24

그 이전 30일

초안에서 가장 크게 줄인 지점. 처음에는 화면 4 를 /devices/:id/week, 화면 6 을 /devices/:id/doses 로 나눠 두 개였다. "이 API 가 무엇을 가져오는가" 로 다시 물으니 둘 다 "그 기기의 복약 건" 이었고, 기간만 달랐다. 라우터 하나로 두 화면이 해결되고 화면 6 은 공짜로 달려온다.

▶ 화면이 쓰는 필드

Device	nickname (헤더)
Dose	id(팝업의 taken 처리) · scheduled_at · status · dispensed_at · taken_at · taken_source
has_more	[더 보기] 버튼을 보일지 판단

medications 는 응답에 오지만 이 화면은 쓰지 않는다. 같은 자원은 어느 API 에서든 같은 모양이라는 원칙 때문 — 안 쓰는 필드가 몇 개 오가는 비용은 감수한다.

▶ 표시 규칙 (상태 → 아이콘)

status	조건	표시
taken	—	✓ 복용 완료
missed	dispensed_at 이 null	✕ 약을 안 꺼냄
missed	dispensed_at 이 있음	⚠ 꺼냈으나 미복용
notified dispensed	—	⌚ 진행중
scheduled	—	· 예정

색만으로 구분하지 않는다. 놓침 두 종류는 색이 같으므로 아이콘(✕ / ⚠)까지 다르게 한다. 적록색약이면 초록/빨강이 구분되지 않는데 이 앱 사용자는 중장년층 보호자가 많다.

알림

● 할아버지 기기

약을 안 꺼내셨어요

08:05

● 할아버지 기기

꺼내셨는데 아직 안 드셨어요

8월 20일 08:08

할아버지 기기

약을 안 꺼내셨어요

8월 17일 08:05

기기

알림 2

안 읽은 항목 — **열은 배경 + 왼쪽 빨간 점 + 굵은 글씨.**
색 하나에만 의존하지 않는다.

```
GET /api/notifications?limit=30
```

화면 열릴 때 · 당겨서 새로고침 · 30초 폴링

```
{
  "notifications": [
    { "id": "412", "device_id": "d3f1a2b4-...", "dose_id": "8821",
      "type": "missed_not_dispensed", "message": "약을 안 꺼내셨어요",
      "created_at": "2026-08-23T23:05:00Z", "read_at": null },
    { "id": "409", "device_id": "d3f1a2b4-...", "dose_id": "8818",
      "type": "missed_not_taken", "message": "꺼내셨는데 아직 안 드셨어요",
      "created_at": "2026-08-19T23:08:00Z", "read_at": null }
  ],
  "devices": [ { "...Device..." } ], // 별명 표시용
  "doses": [ { "...Dose..." } ], // 팝업이 켜
  "unread_count": 2,
  "has_more": false
}
```

화면 2 와 똑같은 패턴 — 배열을 나눠 주고 화면이 device_id / dose_id 로 짝을 맞춘다.

devices 는 이 목록에 등장하는 기기만 온다 (전체 기기 목록이 아니다). 알림 3건이 모두 할아버지 기기 것이라면 devices 는 1개다.

날짜 범위가 아니라 개수(limit)인 이유 — 알림은 드물게 쌓여 "최근 30일" 이 2건일 수도 있다. 요청 파라미터는 목록 성격에 맞춘다. (응답 모양은 그래도 동일하다)

POST /api/notifications/:id/read

항목을 탭했을 때 — 읽음 처리 + 팝업 열기가 동시에 일어난다.

```
// 요청 본문 없음

// 응답 200
{
  "notifications": [ { "...갱신된 Notification..." } ],
  "unread_count": 1
}
```

- **unread_count** 를 같이 돌려주므로 탭하는 즉시 하단 탭 배치가 정확한 숫자로 줄어든다. 목록 전체를 다시 안 불러도 된다
- 이미 읽은 알람에 또 보내도 read_at 은 처음 값 그대로 둔다
- 실패 — 403 FORBIDDEN(내 알람이 아님) / 404 NOT_FOUND

★ 푸시(FCM)와 알림 목록은 별개다. 푸시는 단지 먼지 먼지 끝이라 사본이 남지 않는다(문자를 보내도 내 폰에 발신함이 생기지 않는 것과 같다). 그래서 화면 7 이 읽어갈 **알림 테이블이 서버에 따로 필요하다**.

서버는 missed 판정을 받으면 ① 알림 테이블에 한 줄 저장한다. 나중에 여기에 ② 푸시 발송 한 줄만 추가하면 된다.

8월 20일 (목)

예정 08:00
 꺼냄 08:03
 복용 —
 상태 △ 꺼냈으나 미복용

닫기

먹었어요로 표시

한 번 만들어 세 화면에서 재사용한다. 상태를 App/store 가 들고 있어 화면 4-6-7 이 같은 팝업을 연다.

▶ 열릴 때 — 서버를 부르지 않는다

부모 화면이 이미 들고 있는 Dose 객체를 그대로 쓴다. 화면 4-6 은 doses 배열에서, 화면 7 은 dose_id 로 찾아 넘긴다. 그래서 팝업 전용 API 가 없다.

▶ 세 가지 모습

조건	보여주는 것
status === 'taken'	버튼 대신 ✓ 복용 완료 · 판정 근거 (taken_source → 센서/카메라/보호자 직접 확인)
scheduled_at 이 미래	[먹었어요로 표시] 를 감춘다 "아직 복약 시각이 되지 않았습니다"
그 외 (놓침·진행중)	[닫기] [먹었어요로 표시]

POST /api/doses/:id/taken

[먹었어요로 표시] → 한 번 더 확인한 뒤 호출

```
// 요청 본문 없음 - 무엇으로 바꿀지는 서버가 안다
// status='taken', taken_source='manual', taken_at=현재시각

// 응답 200
{ "doses": [ { "id": "8818", "status": "taken",
               "taken_at": "2026-08-24T05:12:00Z",
               "taken_source": "manual", ... } ] }
```

▶ ★ 처리 후 화면 갱신 — "갈아끼우기"

```
const { doses } = await res.json()
const updated = doses[0]
// 화면이 들고 있는 배열에서 id 가 같은 것만 교체
setDoses(prev => prev.map(d => d.id === updated.id ? updated : d))
```

요청 1번. 목록 전체를 다시 안 불러도 된다. 이미 목록에 있는 항목을 제자리에서 바꾸는 것이라 위치가 안 변한다 — 20일 칸은 여전히 20일 칸이고 내용만 △ → ✓ 로 바뀐다.
 화면 3(기기 등록)은 반대로 "다시 부르기" 를 골랐다. 거기는 새로 생긴 것을 가나다순 어디에 넣을지 몰랐다. 이미 있는 것을 바꾸는가 / 새로 생긴 것을 넣는가 의 차이다.

▶ 실패 응답

HTTP	code	언제
400	DOSE_NOT_DUE	아직 오지 않은 복약 건
403	FORBIDDEN	그 복약 건의 기기가 내 것이 아님
404	NOT_FOUND	그런 복약 건 없음
409	ALREADY_TAKEN	이미 복용 처리됨

미래 복약 건은 서버도 막는다. 화면이 버튼을 감추지만 cur1 로 직접 보낼 수 있다. 화면을 못 믿는 게 서버의 기본자세.

그리고 수동 체크는 안전망이다 — 카메라 판정은 실제로 드셨는데 손목이 프레임 밖이면 못 잡는다. 미탐이 오탐보다 아프므로 보호자가 고칠 길을 둔다.

▶ 디바이스 동작 흐름

- ① 부팅 시 · 주기적으로 스케줄을 받아온다
GET /api/device/schedule
- ② 복약 시각 도달 → 음성 안내
POST /api/device/events status=notified
- ③ 5분 내 IR 감지 없음
POST /api/device/events status=missed
(dispensed_at 없음)
- ④ IR 감지 → 카메라 켜기
POST /api/device/events status=dispensed
- ⑤ 카메라가 5분 내 복용 감지
POST /api/device/events status=taken
taken_source=camera
- 감지 못하면
POST /api/device/events status=missed

5분 타이머는 디바이스가 켜다. 서버는 판정하지 않고 보고를 받아 적기만 한다. 서버가 재러면 계속 시계를 보는 크론이 필요해 복잡해지고, **인터넷이 끊기면 음성 안내조차 못 하게 된다.**

▶ 인증 — 왜 API 키인가

라즈베리파이에는 **브라우저도 사람도 없다.** 이메일/비밀번호를 입력할 수 없으므로, 공장에서 기기마다 **긴 문자열(API 키)**을 심어두고 그걸로 신분을 증명한다. X-Device-Key: dk_9f8e7d6c5b4a3f2e... 표준에 없는 헤더는 X- 로 시작하는 게 관례다.

GET /api/device/schedule

부팅 시 · 주기적으로

```
GET /api/device/schedule
X-Device-Key: dk_9f8e7d6c5b4a...

// 응답 200
{
  "devices": [
    { "id": "d3f1a2b4-...", "timezone": "Asia/Seoul" }
  ],
  "medications": [
    { "name": "혈압약", "dosage": "1정",
      "time": "08:00", "days": [1, 3, 5] }
    ]
}
```

- 약 설정이 없으면 "medications": [] → 아무 안내도 하지 않는다
- Device 객체를 통째로 주지 않는다.** 별명·일련번호·환자 전화번호는 기기에게 불필요한 노출이다. 앱용 API 와 객체 모양을 공유하지 않는 유일한 자리

POST /api/device/events

★ 상태가 바뀔 때마다

```
POST /api/device/events
X-Device-Key: dk_9f8e7d6c5b4a...

{
  "scheduled_at": "2026-08-23T23:00:00Z", // 어느 복약 건인지
  "status": "dispensed",
  "occurred_at": "2026-08-23T23:03:12Z", // 실제로 일어난 시각
  "taken_source": null
}

// 응답 200
{ "doses": [ { "...Dose..." } ] }
```

★ scheduled_at 으로 어느 복약 건인지 특정한다 — 가장 놓치기 쉬웠던 부분

디바이스는 dose_id(8821)를 말할 수 없다. 그 번호는 DB 가 만든 것이고 스케줄 응답에 들어 있지도 않다. **병원 접수번호**와 같다 — 환자는 자기 번호를 모르지만 **"3시 예약이요"** 라고 말할 수 있다.

그래서 파이가 계산할 수 있는 값으로 지목한다. device_id 는 **API 키가 말해준다.**

WHERE device_id = <키에서 알아낸 기기> AND scheduled_at = <요청 값>

초안 { status, occurred_at, taken_source? } 은 셋 다 "어느 건인지" 를 말해주지 않았다. 구현 단계에서 발견했으면 **디바이스 파이썬 코드까지** 고쳐야 했다.

occurred_at 이 따로 있는 이유 — 08:03 에 꺼냈는데 와이파이가 끊겨 11:00 에 보고가 도착할 수 있다. "받은 시각"(11:00)을 적으면 기록이 틀린다. **서버는 받은 시각을 쓰지 않는다.** 오프라인 큐를 붙여 밀린 기록을 한꺼번에 보내도 시각이 안 어긋난다.

- 복약 건이 **아직 없으면 서버가 그때 만든다** — 미리 만들어두는 크론이 필요 없다
- status=taken 이면 taken_source 필수 (ir/camera) · 실패 — 401 INVALID_DEVICE_KEY / 400 VALIDATION_ERROR

요청한 보호자가 `user_devices` 에 그 기기와 연결돼 있는지 매번 확인한다.

확인 없이 `device_id` 만 믿으면, 남의 `id` 를 넣어 보내는 것만으로 다른 사람의 복약 기록이 노출된다.

▶ 뚝리는 시나리오

보호자가 자기 앱에서 개발자도구를 열어

```
GET /api/devices/<남의기기UUID>/doses?from=...&to=...
Authorization: Bearer <자기 토큰>
```

를 보내면?

확인 없이 처리하면
→ 남의 할아버지 복약 기록이 그대로 온다
→ 화면 5 를 부르면 복용 시간을 바꿔버릴 수도 있다

▶ 원칙

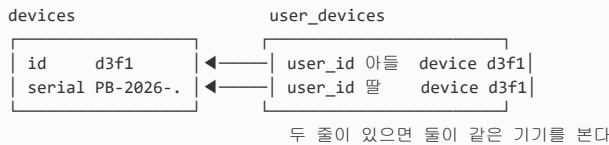
누구인지는 토큰이 말하고, 무엇을 요청했는지는 URL 이 말한다.

서버는 매번 "이 토큰의 주인이 이 기기와 연결돼 있나" 를 `user_devices` 에서 확인한다.

```
SELECT 1 FROM user_devices
WHERE user_id = '<토큰에서 얻은 값>'
AND device_id = '<URL 의 :id>';
```

// 없으면 403 FORBIDDEN

▶ 테이블 구조가 이걸 가능하게 한다



`devices.id` 는 기기를 가리키지 사람을 가리키지 않는다. 아들이 모든 딸이 모든 같은 값이고, 누가 요청했는지는 토큰이 말한다.

사람별 구분은 `user_devices.user_id` 가 한다.

▶ :id 가 기기가 아닌 API

API	확인할 것
POST /api/doses/:id/taken	그 복약 건의 <code>device_id</code> 를 꺼내서 그 기기가 내 것인지 <code>doses</code> → <code>devices</code> → <code>user_devices</code> 로 거슬러 올라간다
POST /api/notifications/:id/read	그 알림의 <code>user_id</code> 가 나인지 <code>notifications</code> 에 <code>user_id</code> 가 직접 있어 한 단계로 끝난다

▶ 403 vs 404

403	선택. 디버깅이 쉽다 — "없는 건가 / 권한이 없는 건가" 가 구분된다
404	"그런 게 존재하는지"조차 안 알려줘 더 안전하다는 견해도 있다

이 프로젝트는 등록코드라는 자물쇠가 따로 있어 존재 여부 노출의 위험이 낮다고 보고 403 으로 통일한다.

▶ 디바이스도 같다

API 키에 묶인 기기의 데이터만 읽고 쓸 수 있다.

POST /api/device/events 의 `scheduled_at` 으로 복약 건을 찾을 때도 "그 키의 기기" 로 범위를 좁혀서 찾아야 한다.

▶ 구현 순서 (spec §9)

5단계	인증 없이 API 를 먼저 구현한다 (<code>user_id</code> 임시 하드코딩)
7단계	Supabase Auth + 이 접근 제어를 붙인다

왜 인증을 나중에 — 인증부터 붙이면 버그가 났을 때 "내 로직이 틀린 건가 / 로그인 안 된 건가" 를 구분할 수 없다. 자물쇠는 문틀을 다 세운 다음에 단다.

▶ 엔드포인트 전체 (10개)

메서드	경로	화면
GET	/api/devices	2
POST	/api/user-devices	3
GET	/api/devices/:id/doses	4 · 6
GET	/api/devices/:id/medications	5
PUT	/api/devices/:id/medications	5
GET	/api/notifications	7
POST	/api/notifications/:id/read	7
POST	/api/doses/:id/taken	공용 팝업
GET	/api/device/schedule	디바이스
POST	/api/device/events	디바이스

모든 화면이 열릴 때 요청 1번. 화면 2 의 배치도 GET /devices 응답에 얹어 해결했다.